Use JSON Lines (`.jsonl`) for training examples: one complete JSON object on each line. It is easy to append, inspect and load using Hugging Face Datasets. A single conversation record can contain a `messages` array: `[{"role":"user","content":"Summarise the policy"},{"role":"assistant","content":"..."}]`.

Do not assume that writing role labels into plain text is sufficient. Chat models expect model-specific templates and special tokens; Transformers provides `tokenizer.apply_chat_template()` to format structured messages for the selected model. Retain the original role-labelled data, applying the right template when preparing training inputs.

For a classification assistant, show both the input and a verified label. For a style assistant, show varied source material and precisely the output format you want. Avoid filling a dataset with fifty nearly identical examples; it encourages memorisation instead of robust performance.

Create examples that teach rather than confuse

Build a small dataset manually before attempting automation. Select thirty representative requests from your target workflow. Cover easy cases, ambiguous questions, exceptions and cases where the correct answer is "I do not have enough information."

Step 1: Define the output contract. For a support ticket classifier, choose fields such as `category`, `priority`, `reason` and `next_action`. Decide permitted categories and write definitions. Without a consistent specification, two people may label the same ticket differently.

Step 2: Produce reference answers. Use the approved source documents, not memory or an unverified chatbot response. Include relevant caveats and source identifiers. Keep examples realistic: Indian addresses, rupee amounts, bilingual queries and locally sold product names where appropriate.



10-MIN DATA QUALITY AUDIT

Take five random documents and five random training examples. First, locate each document's origin and confirm the latest approved version. Check whether extracted tables, dates and negations match the source. Next, inspect metadata for missing page numbers, language labels or access restrictions. Open the five examples and verify that the supposed correct responses are actually supported. Finally, search the processed files for email addresses, phone numbers and obvious secrets, then manually inspect suspicious matches. Record failures in a simple log with issue, source_id, severity and action fields. Repeat after changing extraction scripts. Ten minutes cannot certify a large corpus, but a routine spot check can expose systemic errors before they contaminate hundreds of answers.

The deliverable from this chapter is not an impressive number of gigabytes. It is a clean, permission-aware corpus with traceable sources, plus a small collection of verified examples and untouched evaluation cases. These are the ingredients that let an AI system move beyond generic answers. Next, we will make that knowledge searchable.

Step 3: Review disagreements. Have another person assess a subset independently if possible. When labels differ, improve the rubric before manufacturing more examples. Record edge cases rather than silently deleting them.

Step 4: Validate the file. Ensure each JSONL line parses as JSON, mandatory fields exist, role names are valid and no assistant answer is missing. Load it with `load_dataset("json", data_files="data/training/examples.jsonl")`. This confirms the file can be read; it does not confirm that its answers are correct.

A small trustworthy dataset is an excellent first milestone. You can expand it after seeing where the model actually fails.

Keep test questions out of training

Training and evaluation have different jobs. Training examples show the model what to learn. Validation examples help you compare settings during development. Test examples estimate performance on material the system has not seen. A practical first split might be 80 percent training, 10 percent validation and 10 percent test, though tiny datasets often benefit more from a carefully designed fixed evaluation set.